

Jaypee Institute of Information Technology, Noida

Department of Computer Science & Engineering and IT



Major Project Title: Intern at Evalueserve Pvt. Ltd.

Enrol. No.	Name of Student
22103290	Harshit Tyagi

Course Name: Major Project 2

Program: B. Tech. CS&E

8th Sem

2025 – 2026

CERTIFICATE

This is to certify that the work titled “Intern at Evalueserve Pvt. Ltd.” Submitted by **Harshit Tyagi**, in partial fulfilment for the award of degree of B.Tech of Jaypee Institute of Information Technology, Noida has been carried out under my supervision. This work has not been submitted partially or wholly to any other University or Institute for the award of this or any other degree or diploma.

Signature of Supervisor:

Name of Supervisor: Dr. Prashant Kaushik

Date:

DECLARATION

I hereby declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person nor material which has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Place:

Signature:

Date:

Name:

Enrollment Number:

ACKNOWLEDGEMENT

We have taken efforts in this project. However, it would not have been possible without the kind support and help of many individuals and organizations. We would like to extend our sincere thanks to all of them. We would like to express our gratitude towards the members of IIIT for their kind cooperation and encouragement, which helped us in the completion of this project. We would like to express our special gratitude and thanks to industrial persons for giving us such attention and time. Our thanks and appreciations also go to our colleagues in developing the project and people who have willingly helped us out with their abilities.

Signature:

Harshit Tyagi (22103290)

Date:

ABSTRACT

The rapid growth of digital platforms has transformed the way services such as food ordering are delivered and consumed. Traditional methods of ordering food often involve manual processes, limited accessibility, and inefficiencies in order management. This project, titled **QuickBite** aims to address these challenges by developing a full-stack web-based food ordering system that integrates modern technologies to provide a seamless and efficient user experience.

The primary problem addressed in this project is the need for a structured, secure, and scalable system that allows users to browse food items, manage carts, and place orders while ensuring proper validation and administrative control. Additionally, the project focuses on implementing secure authentication mechanisms and realistic business workflows that reflect real-world applications.

The methodology adopted involves the use of a three-tier architecture, consisting of a React-based frontend, a FastAPI backend, and a PostgreSQL database. The system is designed using a modular approach, with clear separation of concerns between components. Key features include JWT-based authentication, OTP verification for user registration, role-based access control, and a structured order processing workflow where orders are reviewed and approved by an administrator.

The application successfully integrates frontend and backend components, providing a smooth user experience and accurate data processing. Testing confirms that the system handles both valid and invalid scenarios effectively, ensuring robustness and reliability.

Additionally, the use of cloud-based deployment and digital workflows supports environmentally sustainable practices by reducing reliance on physical resources. In conclusion, the **QuickBite** project successfully achieves its objectives by developing a functional and secure full-stack application. It serves as a practical demonstration of modern web development practices and provides a foundation for future enhancements such as payment integration, real-time tracking, and scalability improvements.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
Chapter - 1	Introduction	12-18
	1.1 Background	12-13
	1.2 Problem Definition and Objectives	13-14
	1.3 Operational and Business Relevance	14-15
	1.4 Internship Project Plan	15-17
	1.5 Scope and Limitations	17-18
Chapter - 2	Literature Survey & Technology Landscape	19-25
	2.1 Critical Review of Industry Technologies	19-22
	2.2 Comparative Summary of Methodologies	23-24
	2.3 Identification of Operational and Technical Gaps	24-25
Chapter - 3	System Design & Methodology	26-39
	3.1 Design Considerations	26-28
	3.2 System Architecture and Methodology	28-32
	3.3 Technical Modelling and Metrics	33-36
	3.4 Tool and Technique Selection	37-39
Chapter - 4	Implementation & Testing	40-52
	4.1 Implementation and Execution Details	40-46
	4.2 Experimental and Operational Setup	46-46
	4.3 Testing and Verification	47-51
	4.4 Safety and Security Measures	52-52

Chapter - 5	Results & Discussion	53-59
	5.1 Performance and Outcome Analysis	53-55
	5.2 Achievement of Objectives	55-56
	5.3 Economic and Resource Analysis	56-58
	5.4 Comparative Evaluation with Industry Standards	58-59
Chapter - 6	Conclusion & Future Scope	60-65
	6.1 Summary of Achievements	60-61
	6.2 Professional Impact, Ethics, and Compliance	61-62
	6.3 Limitations	62-63
	6.4 Future Work and Recommendations	63-65
	References	66-67
	Annexure	68-70

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1	Use Case Diagram of QuickBite Project	29
2	Activity Diagram for Order Processing Workflow	30
3	Sequence Diagram for Authentication and Order Processing	31
4	Class Diagram of QuickBite Backend System	32
5	Entity Relationship Diagram of QuickBite Project	34
6	Token-Based Authorization Flow	35
7	Database Model	41
8	Login JWT Token	42
9	OTP Verification Screen	42
10	Home Page UI	43
11	Admin Dashboard	44
12	Order Placed Screen	45
13	Admin Approval Screen	45

LIST OF TABLES

Table No.	Table Name	Page No.
1	Comparative Summary of Technologies and Methodologies Used	24
2	Test Cases for Authentication and User Management Operations	47
3	Test Cases for Cart Management Operations	48
4	Test Cases for Order Processing Operations	48
5	Test Cases for Admin Operations	49
6	Test Cases for Authentication Logic Validation (White Box)	50
7	Test Cases for OTP Verification Logic	50
8	Test Cases for Order Logic Validation	50
9	Test Cases for Admin Control Logic	51
10	Test Cases for Middleware and Authorization Logic	51
11	Performance Metrics of QuickBite System	54
12	Comparison of Objectives and Outcomes	56
13	Bill of Materials (Software Resources)	57
14	Comparison with Existing Solutions and Industry Standards	59

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
API	Application Programming Interface
UI	User Interface
UX	User Experience
IDE	Integrated Development Environment
JS	JavaScript
TS	TypeScript
CSS	Cascading Style Sheets
SQL	Structured Query Language
NoSQL	Not Only SQL
DB	Database
ORM	Object Relational Mapping
CRUD	Create, Read, Update, Delete
JSON	JavaScript Object Notation
HTTP	HyperText Transfer Protocol
REST	Representational State Transfer

JWT	JSON Web Token
OTP	One-Time Password
MFA	Multi-Factor Authentication
CI	Continous Integration
CD	Continous Deployment

CHAPTER – 1

INTRODUCTION

1.1 Background (Motivation and Need)

The rapid digital transformation across industries has significantly increased the demand for scalable, responsive, and secure web applications. Businesses today rely heavily on web platforms to deliver services efficiently, particularly in domains such as food delivery, e-commerce, and on-demand services. As a result, there is a growing need for developers who understand not only frontend design but also backend architecture, database management, and system integration.

The motivation behind this internship was to gain practical exposure to modern full-stack development practices and to understand how different technologies work together to form a complete application. While academic learning provides theoretical knowledge, real-world application development involves dealing with integration challenges, debugging issues, and maintaining structured codebases.

The internship at **Evalueserve, Gurgaon**, under the **Innovation and Technology Centre**, was designed to bridge this gap between theory and practice. The program followed a structured approach, consisting of approximately **60% training and 40% project-based implementation**. The training phase focused on building a strong foundation in technologies such as **HTML, CSS, JavaScript, TypeScript, Angular, React, FastAPI, and SQL**, along with introductory exposure to cloud concepts and modern development tools.

Following the training, project assignments were provided to evaluate the understanding of these technologies. These projects simulated real-world application scenarios, particularly in the **food ordering and delivery domain**, which is widely used and highly relevant in today's digital ecosystem.

Two projects were undertaken:

1. **PizzaDelivery (Frontend Application using Angular)**

A structured frontend application implementing CRUD operations, routing, authentication and user interaction features.

2. QuickBite (Full Stack Application)

A complete web application integrating frontend (React + TypeScript), backend (FastAPI), and database (PostgreSQL), with features such as JWT authentication, admin-user role separation, and cloud-based image handling.

The need for such projects lies in developing the ability to design and implement systems that are not only functional but also follow industry-standard practices such as modular architecture, API-driven communication, and secure authentication mechanisms.

1.2 Problem Definition and Objectives

Problem Definition

The internship was structured as a training and evaluation-driven program, where the primary challenge was not solving a single business problem but applying learned concepts to build complete and functional applications.

The core technical problem can be defined as:

To design and implement frontend and full-stack web applications using modern technologies while ensuring proper integration, structured architecture, and secure communication between system components.

During development, several practical challenges emerged, including:

- Integrating frontend applications with backend APIs
- Managing authentication using JWT tokens
- Handling HTTP errors such as **401 Unauthorized** and **307 redirects**
- Maintaining consistency in API routes and data structures
- Designing scalable and maintainable code architecture

Objectives

The following measurable objectives were defined for the internship projects:

1. **To develop a functional frontend application (PizzaDelivery) using Angular**, implementing:
 - CRUD operations
 - Routing and navigation

- Form validation and user interaction
- 2. To build a full-stack web application (QuickBite) with:**
 - React (TypeScript) frontend
 - FastAPI backend
 - PostgreSQL database integration
- 3. To implement secure authentication mechanisms** using JWT, ensuring:
 - Protected routes
 - Role-based access (admin and user)
- 4. To design and integrate RESTful APIs** for communication between frontend and backend systems.
- 5. To follow structured development practices, including:**
 - Interface based design
 - Service based architecture
 - Modular code organization

These objectives were clearly defined and achieved through guided project implementation and continuous testing.

1.3 Social & Environmental Relevance

The applications developed during the internship, although part of a training program, reflect systems commonly used in real-world scenarios, particularly in the food delivery and online service industry.

Social Relevance

Food delivery platforms have become an essential part of modern urban life. Applications like the ones developed in this internship contribute to:

- **Convenience for users**, allowing them to order food from anywhere
- **Support for local businesses**, enabling restaurants to reach a wider audience

- **Efficient service management**, through digital order tracking and processing The QuickBite application simulates a real-world workflow where:
- Users browse and place orders
- Admin manages and approves orders
- Status updates are communicated back to users

This reflects how modern platforms improve accessibility and service efficiency.

Environmental Relevance

Digital applications also contribute indirectly to environmental sustainability:

- **Reduction in paper usage** through digital menus and receipts
- **Optimized resource management** as systems can track demand and reduce wastage
- **Efficient logistics planning**, which can potentially reduce the fuel consumption in real-world systems.

Although the project is a prototype, it demonstrates how technology can support more efficient and sustainable operations.

1.4 Project Plan

The internship followed a structured plan consisting of training, development, testing, and deployment phases.

1.4.1 Work Breakdown Structure

The work was divided into smaller, manageable tasks:

Phase 1: Training

- Learning frontend technologies (HTML, CSS, JS, Angular)
- Learning backend concepts (FastAPI, databases)
- Understanding TypeScript and API design

Phase 2: Frontend Project (PizzaDelivery)

- UI development using Angular
- Implementation of routing and guards

- Form validation and authentication

Phase 3: Full Stack Project (QuickBite)

- Backend development using FastAPI
- Database design and migration using Alembic
- Frontend development using React
- Integration of frontend and backend
- Debugging errors
- API testing using Swagger
- Deployment on Render

Phase 4: Testing and Deployment

1.4.2 Timeline

The internship was conducted over a period of **six months (February 9 – August 9)**.

- **Month 1:** Training and foundational learning
- **Month 2:** Angular frontend project (PizzaDelivery)
- **Month 3-4:** Full stack development (QuickBite), testing and deployment

1.4.3 Roles and Responsibilities

- **Intern (Self):**
 - Learning and implementing technologies
 - Developing frontend and backend features
 - Testing and debugging applications
- **Trainers/Mentors:**
 - Providing guidance and learning materials
 - Reviewing progress and evaluating projects
 - Assisting in resolving technical issues

The projects were developed individually but under continuous supervision.

1.4.4 Challenges and Risk Management

Several challenges were encountered during the project:

Technical Challenges

- JWT authentication errors (401 issues)
- Routing inconsistencies (307 errors due to trailing slashes)
- API integration issues

Risk Management Strategies

- Systematic debugging and testing
- Using tools like Swagger for API verification
- Maintaining consistent API structure
- Refactoring code for better clarity

Learning Challenges

- Adapting to multiple frameworks simultaneously
- Understanding asynchronous programming

These were addressed through practice and guided support.

1.5 Scope and Limitations

Scope

- The scope of this internship includes:
- Development of frontend and full-stack applications
- Implementation of modern web technologies
- Integration of APIs and database systems
- Application of authentication and validation mechanisms

The project provides a strong foundation in:

- Full-stack development
- API design
- System architecture
- Debugging and testing

Limitations

Despite successful implementation, the project has certain limitations:

1. Training-Oriented Approach

- Projects were designed for evaluation rather than real business deployment

2. Limited Scalability

- Applications were not tested for high user loads

3. Deployment Constraints

- Limited exposure to advanced DevOps practices

4. Feature Limitations

- No payment integration
- No advanced analytics or recommendation systems

5. Time Constraints

- Limited time for adding advanced features due to training focus

CHAPTER – 2

LITERATURE SURVEY

2.1 Critical Review

The development of modern web applications depends on a combination of structured frontend frameworks, efficient backend systems, and reliable database management tools. The understanding of these technologies during the internship was primarily guided by **official documentation and practical exposure**. Since internal training resources are confidential, this section reviews publicly available documentation that aligns with the technologies used in the projects.

2.1.1 Angular Framework

Reference:

[1] Angular Documentation, “Angular - The modern web developer’s platform,” Available: <https://angular.dev/>

Summary:

The Angular documentation presents Angular as a complete frontend framework built on TypeScript, designed for developing scalable and maintainable web applications. One of its defining features is its strong architectural structure, which enforces the use of components, modules, and services. Angular introduces the concept of component-based development, where each part of the user interface is encapsulated into reusable units. This approach improves code organization and allows developers to manage complex applications more effectively. Additionally, Angular’s dependency injection system enables efficient sharing of services across components without tightly coupling them, which enhances flexibility and maintainability. Another significant aspect highlighted in the documentation is Angular’s support for reactive programming and form handling. It provides tools for both template-driven and reactive forms, enabling robust validation and dynamic user interaction. Angular also includes built-in routing capabilities, allowing developers to manage navigation and protect routes using guards. From a practical perspective, Angular is particularly suitable for structured and large-scale applications. However, the documentation also reflects that mastering Angular requires understanding

multiple concepts such as TypeScript, RxJS, and modular architecture, which can increase the initial learning effort.

2.1.2 React Framework

Reference:

[2] React Documentation, “React – A JavaScript library for building user interfaces,” Available: <https://react.dev/>

Summary:

React is described in its official documentation as a declarative JavaScript library focused on building user interfaces using reusable components. Unlike full-fledged frameworks, React provides flexibility by allowing developers to choose their own tools for routing, state management, and styling. A key concept in React is the virtual DOM, which improves performance by minimizing direct updates to the actual DOM. Instead of reloading the entire interface, React selectively updates only the components that have changed, resulting in faster rendering and improved user experience. The documentation also introduces hooks, which allow functional components to manage state and lifecycle events without requiring class-based components. This simplifies development and makes code more readable.

React’s design encourages component reusability and separation of concerns, making it easier to maintain large applications. However, due to its flexible nature, developers must define their own project structure, which can sometimes lead to inconsistencies if not managed properly. Overall, React is well-suited for building dynamic and interactive interfaces, especially when combined with TypeScript for type safety.

2.1.3 FastAPI Framework

Reference:

[3] FastAPI Documentation, “FastAPI – High performance web framework,” Available: <https://fastapi.tiangolo.com/>

Summary:

FastAPI is described as a modern Python framework designed for building APIs with high performance and minimal code complexity. One of its most notable features is its reliance on Python type hints, which are used for automatic data validation and serialization through Pydantic models. The documentation emphasizes FastAPI’s support for asynchronous

programming, allowing the server to handle multiple requests concurrently. This makes it highly efficient for applications that involve frequent I/O operations such as database queries or API calls.

Another important feature is the automatic generation of interactive API documentation (Swagger UI and ReDoc). This simplifies testing and helps developers understand API endpoints without additional configuration.

FastAPI promotes a structured approach to backend development by encouraging the use of routers, schemas, and modular components. This aligns well with industry practices for building scalable backend systems.

However, effective use of FastAPI requires a clear understanding of asynchronous programming and proper project structuring, which may present a learning challenge for beginners.

2.1.4 PostgreSQL Database System

Reference:

[4] PostgreSQL Documentation, “PostgreSQL Official Documentation,”

Available: <https://www.postgresql.org/docs/>

Summary:

PostgreSQL is presented as a powerful, open-source relational database system that supports advanced data management capabilities. The documentation highlights its ability to handle complex queries, maintain data integrity, and ensure reliable transaction processing.

One of the key strengths of PostgreSQL is its compliance with ACID (Atomicity, Consistency, Isolation, Durability) properties, which ensures that database operations are executed reliably even in cases of system failure. It also supports indexing and query optimization techniques, improving performance for large datasets.

The relational structure of PostgreSQL allows data to be organized into tables with defined relationships, making it suitable for applications that require structured data management. In practical applications, PostgreSQL is widely used due to its scalability and robustness. However, it requires proper schema design and query optimization to achieve optimal performance.

2.1.5 SQLAlchemy ORM and Alembic

References:

[5] SQLAlchemy Documentation, Available: <https://docs.sqlalchemy.org/>

[6] Alembic Documentation, Available: <https://alembic.sqlalchemy.org/>

Summary:

SQLAlchemy is described as a Python-based ORM (Object Relational Mapper) that allows developers to interact with databases using Python objects instead of writing raw SQL queries. This abstraction simplifies database operations and improves code readability.

The documentation explains how SQLAlchemy maps database tables to Python classes, enabling developers to perform operations such as insertion, updating, and querying using object-oriented syntax. This reduces the likelihood of errors and makes the codebase easier to maintain. Alembic, which is built on top of SQLAlchemy, is used for database migration management. It allows developers to track changes in database schema and apply them incrementally. This is particularly useful in collaborative environments where multiple developers may be working on the same database. Together, SQLAlchemy and Alembic provide a structured approach to database management, ensuring consistency and version control. However, setting up migrations and understanding ORM concepts may require additional learning effort.

2.1.6 TypeScript

References:

[7] TypeScript Documentation, Available: <https://www.typescriptlang.org/docs/>

[8] Medium, “Why TypeScript is Better for Large Applications,” Available: <https://medium.com/>

Summary:

TypeScript is defined as a strongly typed superset of JavaScript, designed to improve code quality and maintainability. The official documentation explains that TypeScript introduces static typing, allowing developers to detect errors during development rather than at runtime.

One of the most important features of TypeScript is the use of interfaces, which define the structure of data objects. This ensures consistency across different parts of the application and reduces the risk of unexpected errors.

TypeScript also supports advanced features such as generics and type inference, which make it suitable for large-scale applications. Additional insights from technical articles emphasize that TypeScript improves collaboration in team environments by making code more predictable and easier to understand.

However, adopting TypeScript requires an initial learning effort, especially for developers who are familiar only with JavaScript.

2.2 Summary Table

The following table provides a comparative summary of the technologies and methodologies used:

Technology	Methodology	Efficiency	Cost	Use Case	Limitations
Angular	Component-based, structured framework	High for large apps	Open-source	Enterprise frontend apps	Steep learning curve
React	Component-based, flexible library	High UI performance	Open-source	Dynamic UI apps	Less enforced structure
FastAPI	RESTful API, async backend	High performance	Open-source	API development	Requires async understanding
PostgreSQL	Relational database	High reliability	Open-source	Data storage	Requires schema design
SQLAlchemy	ORM-based DB interaction	Moderate	Open-source	Backend abstraction	Learning complexity
Alembic	Migration tool	High consistency	Open-source	DB version control	Requires setup
Swagger	API testing tool	High debugging efficiency	Free/paid	API testing	Not used in production

TypeScript	Static typing system	Improves code quality	Open-source	Large applications	Initial learning curve
------------	----------------------	-----------------------	-------------	--------------------	------------------------

Table 1: Comparative Summary of Technologies and Methodologies — comparing Angular, React, FastAPI, PostgreSQL, and other tools across efficiency, cost, use case, and limitations

Analysis of Comparison

From the table, it can be observed that all selected technologies are **open-source and widely adopted**, making them suitable for modern application development. Angular provides strong structure, while React offers flexibility. FastAPI ensures high performance in backend systems, and PostgreSQL provides reliable data storage.

However, most technologies introduce a **learning curve**, especially when used together in a fullstack system. Proper understanding of each layer is necessary to ensure smooth integration.

2.3 Identification of Research Gaps

Despite the availability of comprehensive documentation and modern tools, several gaps were identified during practical implementation.

2.3.1 Gap Between Documentation and Practical Implementation

Official documentation provides a strong theoretical foundation but often lacks real-world debugging scenarios. For example:

- Handling JWT authentication errors
- Managing API inconsistencies
- Debugging integration issues

These challenges are usually learned through hands-on experience rather than documentation.

2.3.2 Integration Complexity

While individual technologies are well-documented, integrating them together introduces complexity:

- Frontend-backend communication issues
- CORS configuration problems
- Data format mismatches

This highlights a gap in understanding how systems interact as a whole.

2.3.3 Authentication Challenges

JWT-based authentication, although widely used, presents challenges such as:

- Token validation errors
- Improper header handling
- Debugging 401 Unauthorized issues

Documentation explains the concept but does not always address common implementation mistakes

2.3.4 Routing and API Design Issues

A practical issue observed was related to **HTTP 307 redirects**, caused by inconsistent use of trailing slashes in API routes. This indicates a gap in understanding API design conventions.

2.3.5 Limited Coverage of Production Systems

Most documentation focuses on development-level implementation. Topics such as:

- Large-scale deployment
- Load balancing
- System scalability are not deeply covered in beginner-level resources.

2.3.6 State Management Complexity

Managing application state in frontend frameworks becomes complex as applications grow. While tools like Context API, Redux toolkit exists, they require careful design to avoid inconsistencies.

CHAPTER – 3

SYSTEM DESIGN & METHODOLOGY

3.1 Design Considerations

The design of the QuickBite system was not limited to technical implementation alone. Multiple realworld factors such as economic feasibility, environmental impact, usability, ethical responsibility, and system sustainability were considered to ensure a well-rounded and practical solution.

3.1.1 Economic Considerations

The system was developed using **open-source technologies**, which significantly reduces the overall cost of development and deployment. Frameworks such as React, FastAPI, and PostgreSQL are freely available and widely supported.

Additionally, deployment was performed using platforms like **Render**, and media storage was handled through **Cloudinary**, both of which provide free-tier services. This makes the solution economically viable for small-scale startups or student projects without requiring heavy investment in infrastructure.

3.1.2 Environmental Considerations

Although software systems do not directly impact the environment, the design indirectly contributes to sustainability by:

- Reducing dependency on physical resources such as printed menus and receipts
- Promoting digital workflows
- Utilizing cloud-based infrastructure instead of physical servers

Efficient API design also ensures that unnecessary server load is minimized, which contributes to optimized resource utilization.

3.1.3 Social Considerations

The QuickBite system reflects real-world food delivery applications, which are widely used in modern society. It enhances:

- **User convenience**, allowing easy browsing and ordering

- **Accessibility**, by providing a simple and structured interface
- **Operational efficiency**, through streamlined order processing

The inclusion of separate user and admin roles ensures that responsibilities are clearly defined, which mirrors real-world business operations.

3.1.4 Political and Regulatory Considerations

Although the project is not deployed in a regulated production environment, certain basic compliance practices were followed:

- Secure handling of user authentication
- Avoidance of exposing sensitive backend logic
- Proper validation of user inputs

In a real-world scenario, additional compliance with data protection regulations would be necessary.

3.1.5 Ethical Considerations

Ethical considerations were incorporated into the system design through:

- Secure password handling using hashing
- JWT-based authentication to protect user data
- OTP-based email verification to ensure account authenticity

The system avoids storing plain-text sensitive data and ensures that access to protected routes is restricted.

3.1.6 Health and Safety Considerations

From a system reliability perspective, health and safety relate to ensuring that the application behaves predictably and avoids failures:

- Input validation prevents invalid or harmful data
- Error handling ensures controlled responses
- Authentication prevents unauthorized access

A stable system reduces user frustration and improves overall experience.

3.1.7 Sustainability Considerations

The system was designed with long-term maintainability in mind:

- Modular architecture allows easy updates
- Separation of frontend and backend enables independent scaling
- Reusable components reduce development effort

This ensures that the application can be extended in the future without major restructuring.

3.2 System Architecture and Methodology

The development of the QuickBite application followed a **structured full-stack methodology**, integrating frontend, backend, and database layers through well-defined workflows and communication patterns. The system was designed using a modular approach to ensure clarity, scalability, and maintainability.

3.2.1 Design Approach

The overall design approach is based on the following principles:

- **Separation of concerns** between frontend, backend, and database
- **Component-based frontend architecture** using React
- **Service-based communication** through REST APIs
- **Modular backend structure** using FastAPI
- **Role-based access control** for user and admin operations

This structured approach ensures that each part of the system operates independently while maintaining smooth integration.

3.2.2 Use Case Representation

The system supports two primary actors:

- **User** – interacts with the application to browse food, manage cart, and place orders
- **Admin** – manages food items and processes user orders

The use case diagram provides a high-level view of all functionalities available to both actors and illustrates how different operations are interconnected.

This diagram highlights core system interactions such as registration, login, cart management, order placement, and admin approval, forming the foundation of system functionality.

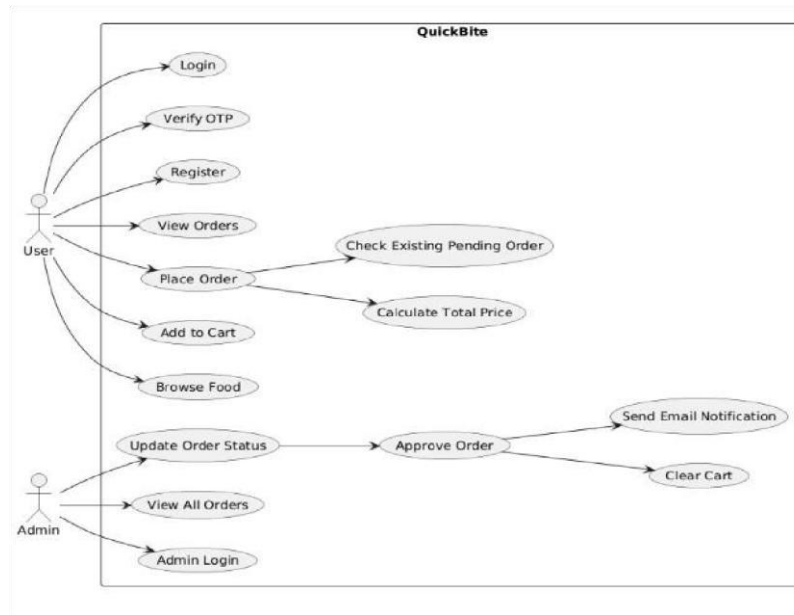


Figure 1: Use Case Diagram of QuickBite Project: showing interactions between User and Admin actors across all system functionalities

3.2.3 System Architecture

The QuickBite system follows a **three-tier architecture**:

1. **Presentation Layer (Frontend):** Developed using React with TypeScript, responsible for user interaction and UI rendering.
2. **Application Layer (Backend):** Implemented using FastAPI, handling business logic, authentication, and API routing.
3. **Data Layer (Database):** PostgreSQL is used to store structured data such as users, food items, cart entries, and orders.

The communication between these layers is achieved through RESTful APIs, ensuring a clean separation between client and server.

3.2.4 Activity Flow of the System

The operational workflow of the system is represented through an activity diagram, which captures the step-by-step execution of user and admin actions.

The flow begins with user registration and OTP verification, followed by login, browsing food items, adding items to the cart, and placing an order. The backend then processes the request by validating cart contents and checking for existing pending orders.

Once an order is created, the admin reviews and updates its status. Upon approval, the system clears the user's cart and sends an email notification. This structured flow ensures controlled and reliable system behavior .

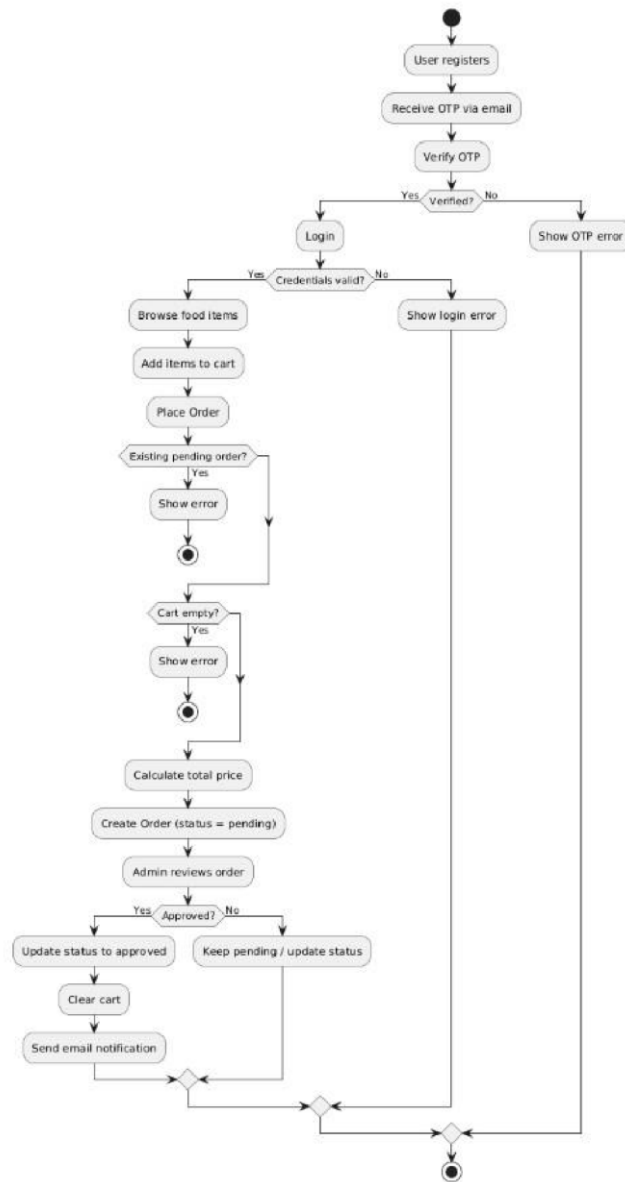


Figure 2: Activity Diagram for Order Processing Workflow — illustrating step-by-step flow from user login to order approval and email notification

3.2.5 Sequence of Operations (API Interaction)

The interaction between different components of the system is best understood through a sequence diagram, which illustrates how requests and responses flow between the frontend, backend, and database.

The sequence begins with user authentication, where credentials are validated and a JWT token is generated. This token is then used for subsequent requests such as placing orders.

When a user places an order, the frontend sends a request with the JWT token to the backend. The backend validates the token, processes the request, interacts with the database, and returns the response. Similarly, admin actions trigger backend updates and email notifications.

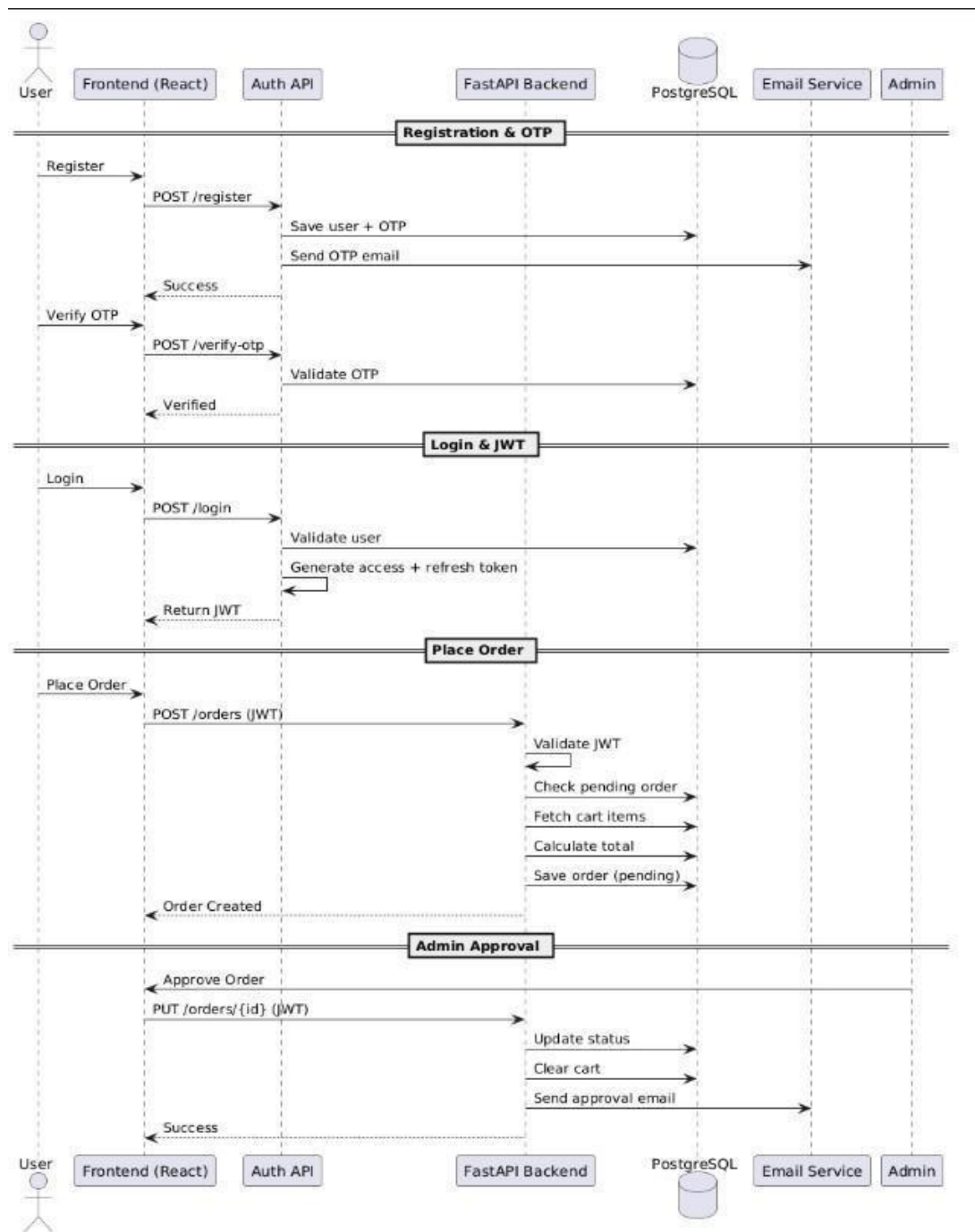


Figure 3: Sequence Diagram for Authentication and Order Processing — depicting request-response flow between frontend, backend, and database

3.2.6 Class Structure and System Components

The internal structure of the system is represented using a class diagram, which shows how different entities and components are organized.

The class diagram includes core entities such as User, Food, Cart, and Order, along with their relationships. It also represents supporting components such as authentication services and email utilities.

This diagram helps in understanding how data is structured and how different components interact within the backend system.

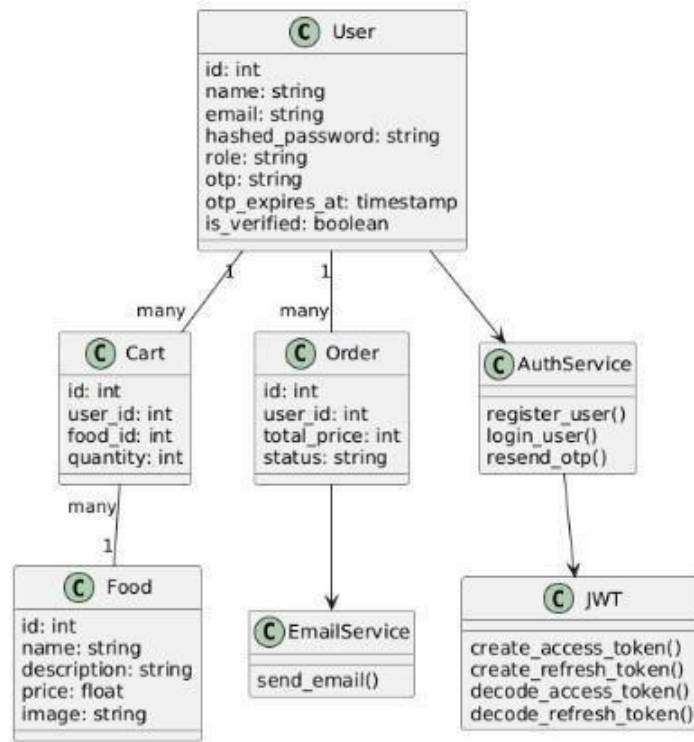


Figure 4: Class Diagram of QuickBite Backend System — representing core entities (User, Food, Cart, Order) and their relationships

3.2.7 Summary of Methodology

The methodology adopted for the QuickBite system ensures:

- Clear separation between system layers
- Efficient communication using REST APIs
- Secure authentication using JWT
- Structured data handling through relational modelling

The use of diagrams such as use case, activity, sequence, and class diagrams provides a comprehensive understanding of both system behavior and structure.

3.3 Modelling

Modelling in the QuickBite system focuses on representing the structure, behavior, and validation of data and processes. Instead of relying on complex mathematical or biological models, the system uses **logical, relational, and computational models** to ensure correctness, consistency, and reliability. These models help validate system functionality such as authentication, order processing, and data integrity.

3.3.1 Data Modelling (Relational Model)

The system follows a **relational data model**, where entities such as User, Food, Cart, and Order are structured into tables with defined relationships.

This model ensures:

- Data consistency through foreign key relationships
- Efficient querying and storage
- Elimination of redundancy

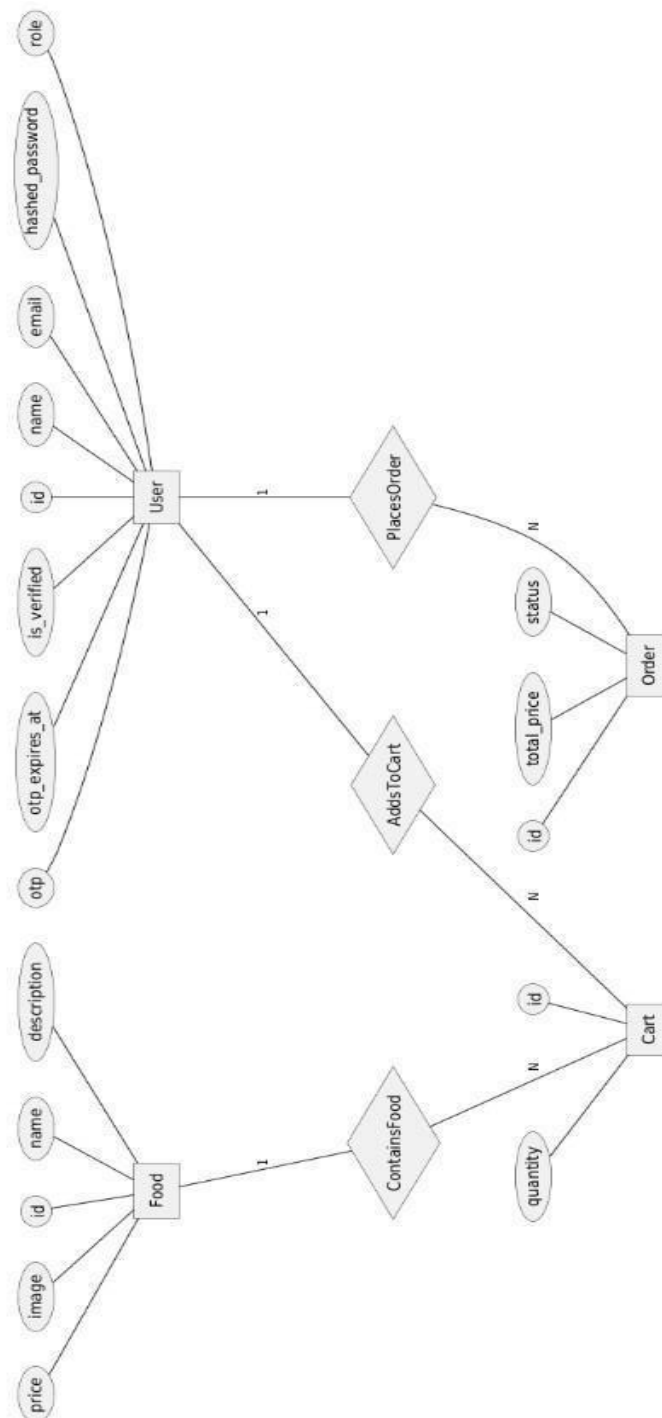


Figure 5: Entity Relationship (ER) Diagram of QuickBite Project — showing relational structure and foreign key associations between database tables

The ER diagram validates that:

- Each cart entry is linked to a valid user and food item
- Each order is associated with a specific user
- Relationships follow real-world constraints

3.3.2 Authentication Model (JWT-Based Validation)

Authentication in the system is based on a **token validation model using JSON Web Tokens (JWT)**.

Token=Header+Payload+Signature

This model ensures:

- **Integrity:** Signature verifies that the token has not been altered
- **Authentication:** Payload contains user identity
- **Authorization:** Role-based access is enforced

The decoding process validates the token before allowing access to protected routes, ensuring secure communication.

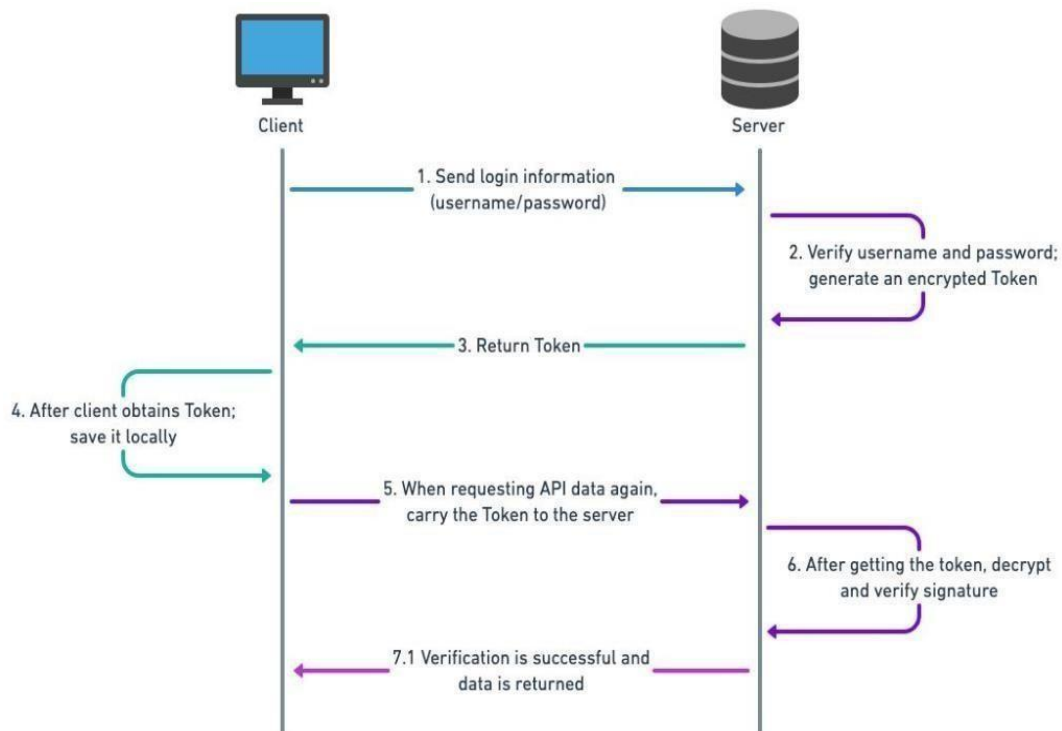


Figure 6: Token-Based Authorization Model — illustrating JWT structure (Header + Payload + Signature) and validation flow

3.3.3 Order Pricing Model (Computation Model)

The total price of an order is calculated using an aggregation model:

$$Total\ Price = \sum_{i=1}^n (Price_i \times Quantity_i)$$

This equation validates:

- Correct computation of order value
- Accurate billing based on cart contents

- Prevention of inconsistencies in pricing

The model ensures that every order reflects the exact sum of selected items.

3.3.4 State Transition Model (Order Lifecycle)

The order system follows a state transition model, where an order progresses through predefined states:

Pending → Approved → Delivered This

model ensures:

- Controlled workflow execution
- Prevention of invalid transitions
- Clear tracking of order status

The validation condition can be expressed as:

- An order can only move to “approved” if it exists and is in “pending” state
- Cart is cleared only when status becomes “approved”

3.3.5 Validation Model (Schema-Based Data Validation)

The system uses **Pydantic-based validation**, which acts as a rule-based model:

- Input data → validated against schema
- Invalid input → rejected
- Valid input → processed This ensures:
- Data integrity
- Prevention of runtime errors
- Consistency in API behavior

3.3.6 Constraint-Based Model

The system also applies logical constraints such as:

- A user cannot place a new order if a pending order already exists
- Cart must not be empty before placing an order
- Only admins can update order status

These constraints act as validation rules to maintain system correctness.

3.4 Tool and Technique Selection

The selection of tools and technologies for the QuickBite system was based on multiple factors such as performance, scalability, ease of development, cost efficiency, and alignment with industry standards. The chosen tools collectively support the development of a robust, secure, and maintainable full-stack application.

3.4.1 Frontend Technologies

React (with TypeScript)

React was selected as the primary frontend library due to its **component-based architecture**, which allows the application to be divided into reusable and manageable UI components. This improves code maintainability and scalability.

TypeScript was used along with React to introduce **static typing**, which helps in early detection of errors and ensures consistency in data structures across the application. The use of interfaces aligns with a structured development approach, making the codebase easier to understand and extend. Additionally, React provides efficient rendering through its virtual DOM, which enhances application performance.

Material UI

Material UI was chosen as the design framework for building the user interface. It provides **predesigned and customizable components**, which significantly reduces development time while ensuring a consistent and professional look.

The use of Material UI also improves user experience by maintaining responsive and accessible design patterns.

3.4.2 Backend Technologies

FastAPI

FastAPI was selected as the backend framework due to its **high performance and simplicity in API development**. It supports asynchronous programming, allowing efficient handling of multiple requests.

One of the key advantages of FastAPI is its use of Python type hints, which enables automatic request validation and API documentation. This reduces development effort and improves reliability. The framework also supports modular routing, which aligns with the structured design approach used in the project.

Pydantic

Pydantic was used for data validation and schema definition. It ensures that all incoming data is validated against predefined structures before processing.

This improves:

- Data integrity
- Error handling
- System reliability

JWT (JSON Web Token)

JWT was chosen as the authentication mechanism due to its **stateless and secure nature**. It allows user authentication without storing session data on the server.

The use of access and refresh tokens enhances security by:

- Preventing unauthorized access
- Allowing controlled session management
- Supporting scalable system design

3.4.3 Database Technologies

PostgreSQL

PostgreSQL was selected as the database due to its **reliability, scalability, and strong support for relational data models**. It ensures data consistency through ACID properties and supports complex queries.

The relational structure of PostgreSQL is well-suited for managing entities such as users, orders, and cart items.

SQLAlchemy (ORM)

SQLAlchemy was used to interact with the database through an object-oriented approach. It simplifies database operations by mapping tables to Python classes.

This improves:

- Code readability
- Maintainability
- Developer productivity

Alembic

Alembic was used for database schema migration and version control. It allows incremental updates to the database structure without affecting existing data.

This is essential for maintaining consistency across different development stages.

3.4.4 Development and Deployment Tools

Swagger

Swagger was used for API testing and debugging. It allows developers to send HTTP requests and analyze responses, which helps in verifying backend functionality before frontend integration.

Git

Git was used for version control, enabling efficient tracking of code changes and collaboration. It ensures that different versions of the project can be managed effectively.

Render

Render was chosen as the deployment platform due to its ease of use and support for full-stack applications. It allows quick deployment of backend services without complex configuration.

Cloudinary

Cloudinary was used for storing and managing images. It provides cloud-based storage and optimized delivery of media files, reducing the load on the backend server.

3.4.5 Justification of Overall Technology Stack

The selected technology stack provides several advantages:

- **Cost Efficiency:** All tools used are open-source or provide free-tier services
- **Scalability:** Modular architecture supports future expansion
- **Performance:** FastAPI and PostgreSQL ensure efficient backend operations
- **Maintainability:** Structured design and TypeScript improve code quality
- **Security:** JWT authentication and validation mechanisms protect user data

The combination of these tools reflects modern industry practices and ensures that the system is both practical and adaptable for real-world applications.

CHAPTER – 4

IMPLEMENTATION & TESTING

4.1 Development Details

The development of the QuickBite application followed a structured and incremental approach, where each module was implemented, tested, and integrated step by step. The system was built as a **fullstack web application**, consisting of a React (TypeScript) frontend, FastAPI backend, and PostgreSQL database.

Step 1: Project Setup and Environment Configuration

The development process began with setting up separate environments for frontend and backend:

- Backend initialized using FastAPI
- Database configured using PostgreSQL and SQLAlchemy
- Frontend initialized using React with Vite and TypeScript

Dependencies were installed, and project structure was organized into modules such as models, schemas, routers, and services.

Step 2: Database Design and Models Implementation

The next step involved designing the database schema and implementing models for:

- User
- Food
- Cart
- Order

Relationships were defined using foreign keys, ensuring proper linkage between entities. Alembic was used for managing database migrations.


```

food.py 1 X
QuickBiteBackend > app > models > food.py > ...
1 from app.core.database import Base
2 from sqlalchemy import Column, Integer, String, Float, text
3
4 class Food(Base):
5     __tablename__ = "foods"
6
7     id = Column(Integer, primary_key=True, nullable=False, index=True)
8     name = Column(String, unique=True, nullable=False)
9     description = Column(String, unique=True, nullable=False)
10    price = Column(Float, nullable=False)
11    image = Column(String, nullable=False)
12

```

```

order.py 1 X
QuickBiteBackend > app > models > order.py > ...
1
2 from sqlalchemy import Column, Integer, String, ForeignKey, Index, text
3 from app.core.database import Base
4
5 class Order(Base):
6     __tablename__ = "orders"
7
8     id = Column(Integer, primary_key=True, index=True)
9     user_id = Column(Integer, ForeignKey("users.id"), index=True)
10    total_price = Column(Integer, server_default="0")
11    status = Column(String, index=True, server_default="PENDING")
12
13
14

```

```

user.py 2 X
QuickBiteBackend > app > models > user.py > User
1 from app.core.database import Base
2 from sqlalchemy import Column, Integer, String, TIMESTAMP, Boolean, text, Float
3 from sqlalchemy.orm import relationship
4
5 class User(Base):
6     __tablename__ = "users"
7
8     id = Column(Integer, primary_key=True, nullable=False)
9     name = Column(String, nullable=False)
10    email = Column(String, unique=True, nullable=False)
11    hashed_password = Column(String, nullable=False)
12    role = Column(String, nullable=False, server_default="user")
13    otp = Column(String, nullable=True)
14    otp_expires_at = Column(TIMESTAMP(timezone=True), nullable=True)
15    is_verified = Column(Boolean, server_default=text('false'), default=False)

```

```

cart.py 1 X
QuickBiteBackend > app > models > cart.py > ...
1 from app.core.database import Base
2 from sqlalchemy import Column, Integer, ForeignKey
3
4 class Cart(Base):
5     __tablename__ = "carts"
6
7     id = Column(Integer, primary_key=True, nullable=False)
8     user_id = Column(Integer, ForeignKey("users.id"), index=True)
9     food_id = Column(Integer, ForeignKey("foods.id"), index=True)
10    quantity = Column(Integer)
11
12

```

Figure 7: Database Model

Step 3: Authentication System (JWT + OTP)

A secure authentication system was implemented with the following features:

- User registration with email and password
- OTP generation and email verification
- Login system with JWT access and refresh tokens
- Middleware for token validation

This step ensured secure access to protected routes and role-based authorization.

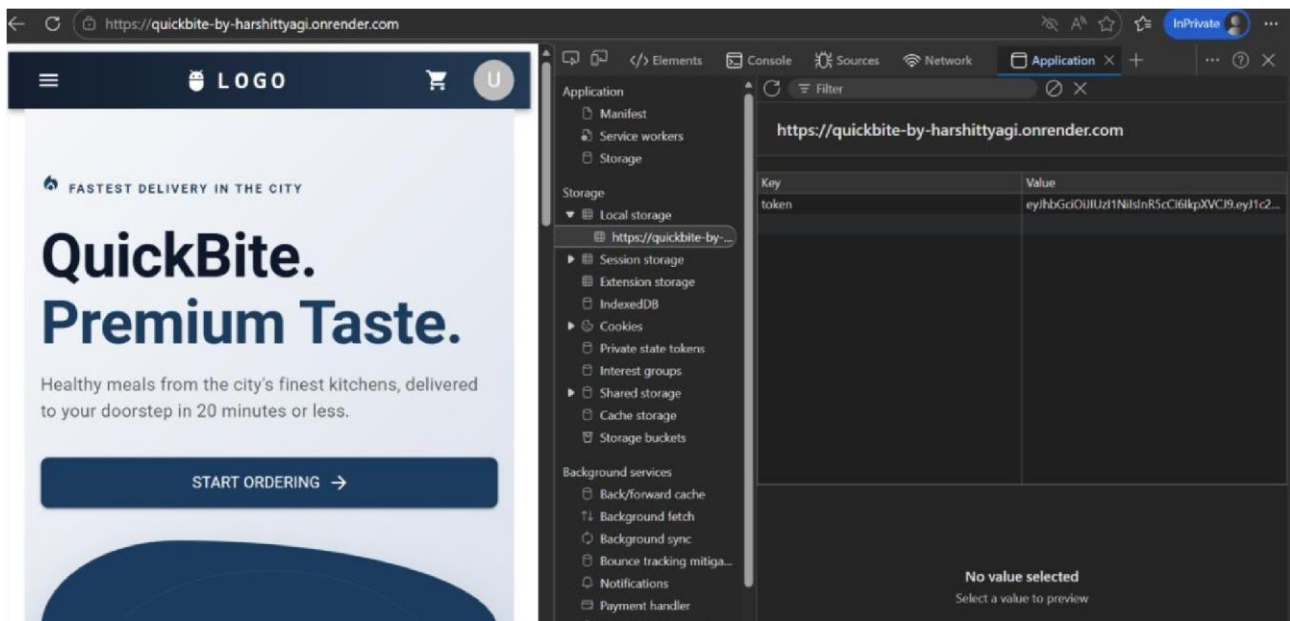


Figure 8: JWT Token Generated on Successful Login — screenshot showing access token returned by the backend

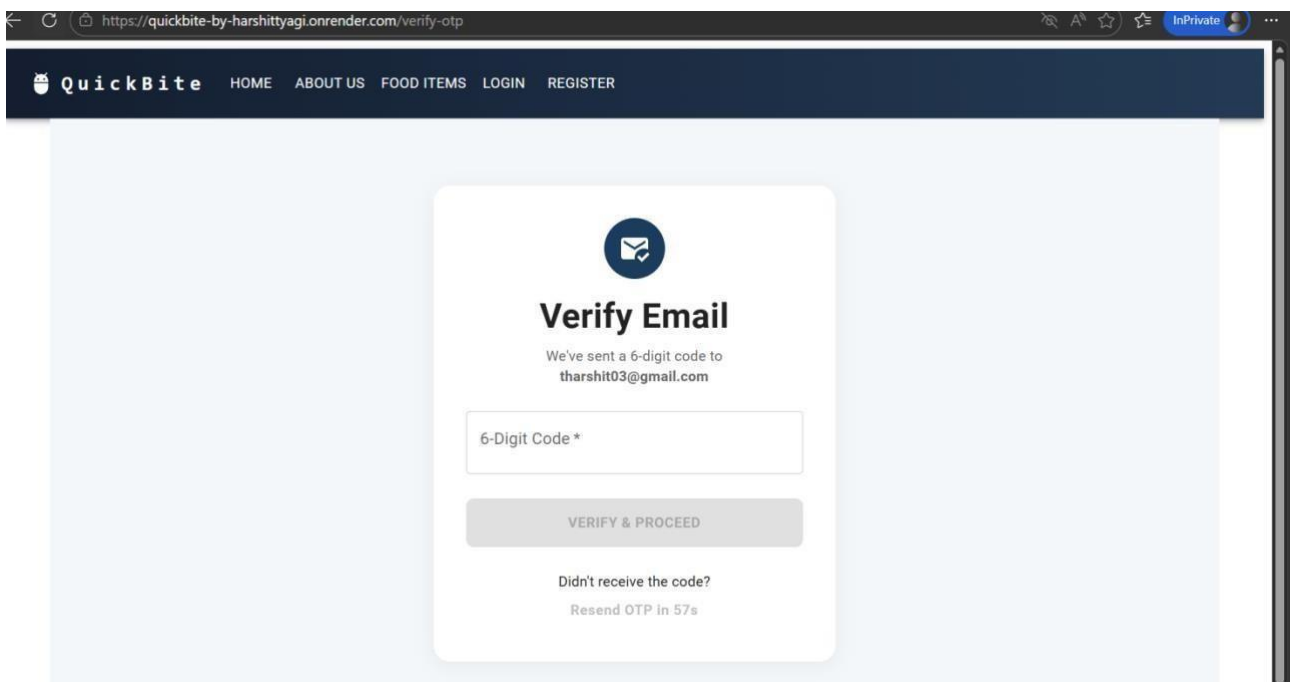


Figure 9: OTP Verification Screen — screenshot showing email-based OTP verification during user registration

Step 4: API Development (Backend Logic)

RESTful APIs were developed using FastAPI for handling:

- User authentication
- Food item management
- Cart operations
- Order placement and status updates Special logic was implemented for:

- Preventing duplicate pending orders
- Calculating total order price
- Role-based access control (user/admin)

Step 5: Frontend Development

The frontend was developed using React with a structured approach:

- Pages divided into user and admin sections
- Context API used for state management
- Service-based API calls implemented
- Material UI used for styling Two layouts were implemented:
- Main layout (user)
- Admin layout

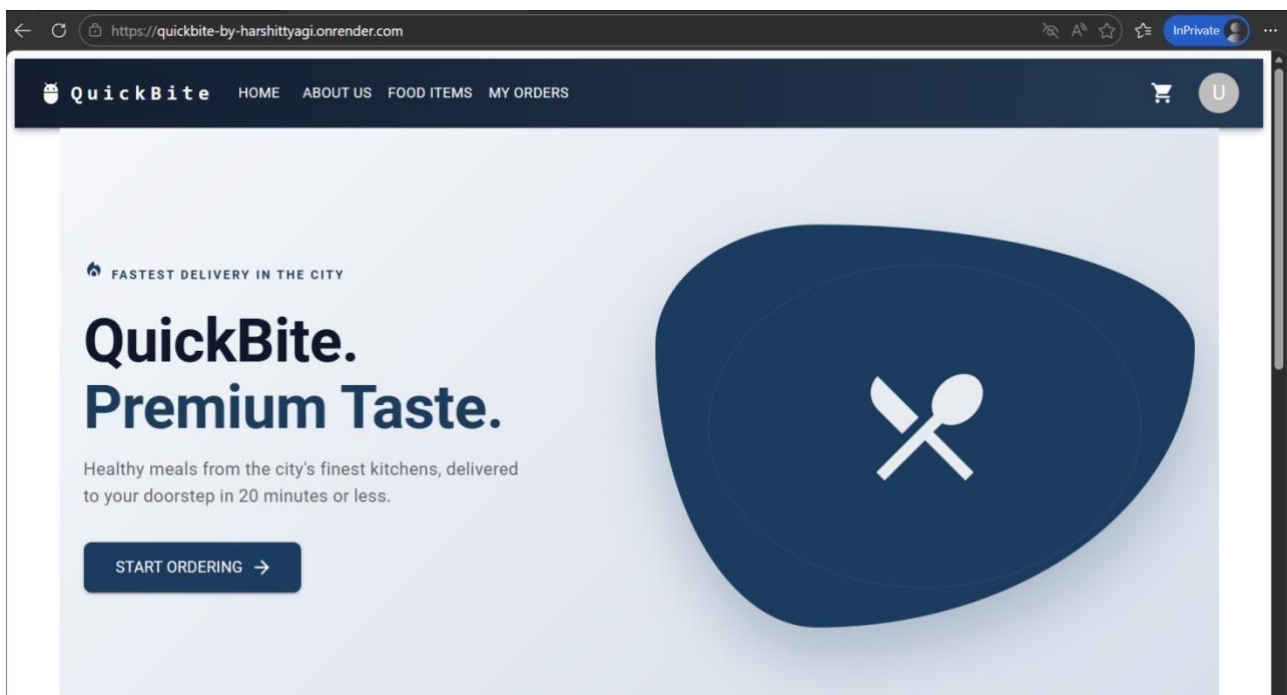


Figure 10: Home Page UI — screenshot of the user-facing frontend interface built using React and Material UI

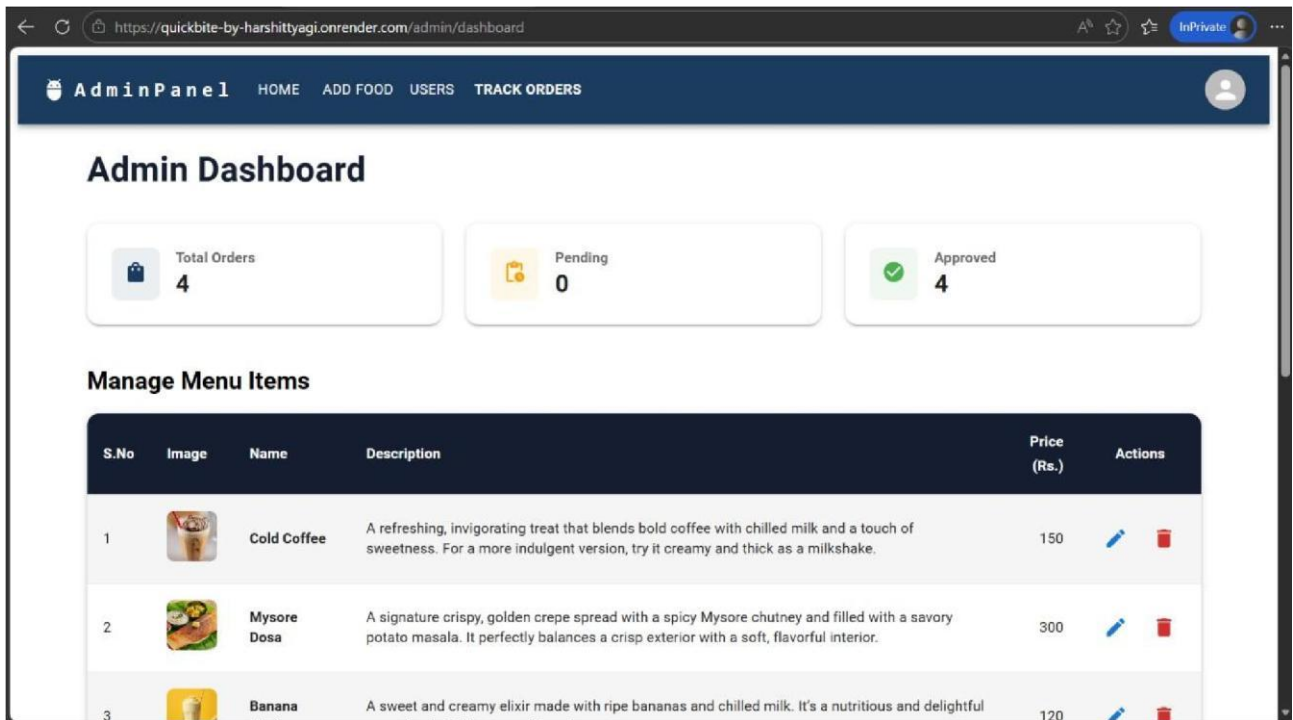


Figure 11: Admin Dashboard — screenshot of the admin panel for managing food items and orders

Step 6: Integration of Frontend and Backend

After independent development, frontend and backend were integrated:

- API endpoints connected via service layer
- JWT token stored and sent in requests
- Dynamic data rendering implemented

This step ensured real-time interaction between user interface and backend logic.

Step 7: Order Processing Workflow

The complete order lifecycle was implemented:

1. User adds items to cart
2. Places order
3. Backend validates cart and calculates total
4. Order stored with “pending” status
5. Admin approves order
6. Cart is cleared
7. Email notification sent

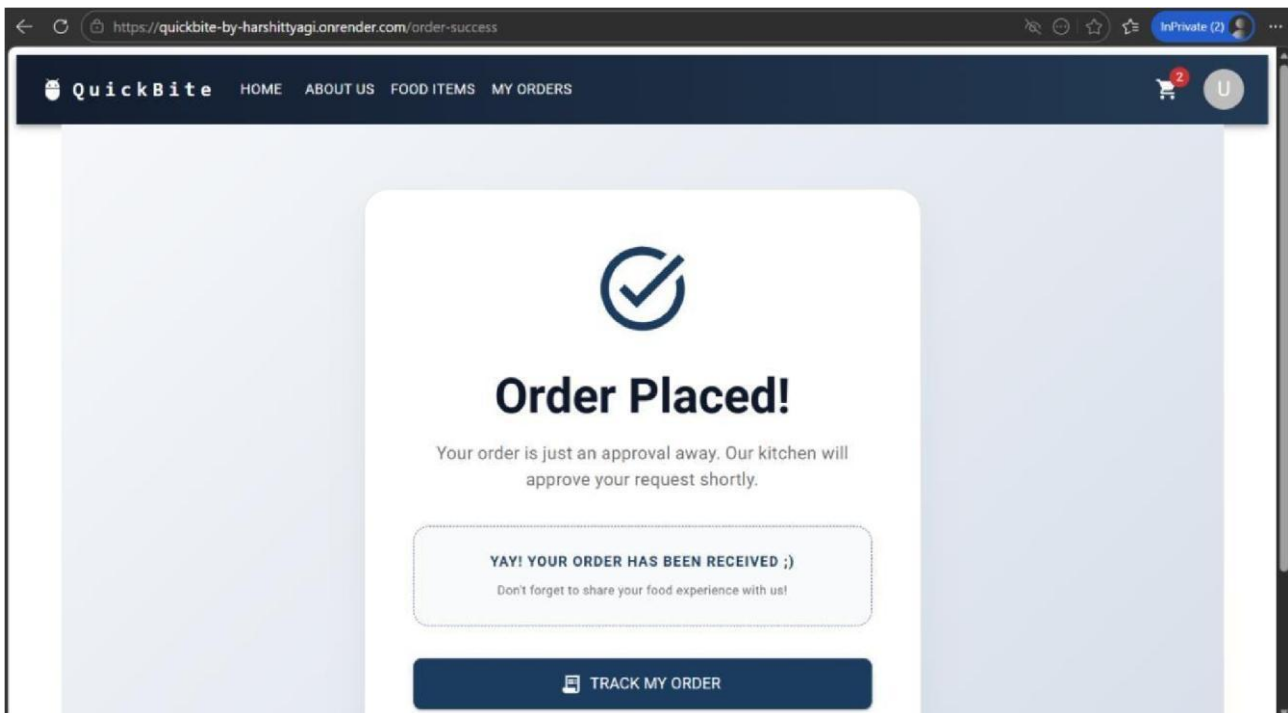


Figure 12: Order Placed Confirmation Screen — screenshot showing successful order placement with pending status

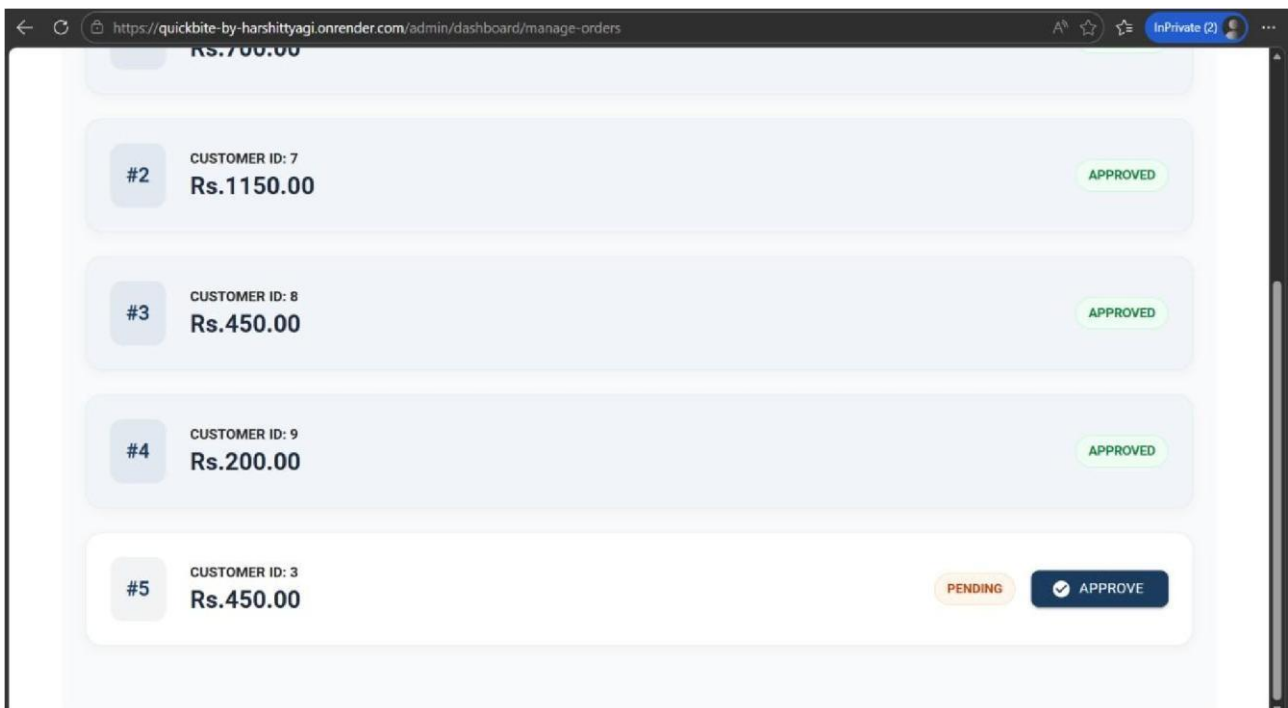


Figure 13: Admin Order Approval Screen — screenshot showing admin interface for approving user orders

Step 8: Deployment

The application was deployed using:

- Backend on Render
- Image storage using Cloudinary

Deployment ensured that the application is accessible and functional in a real-world environment.

4.2 Experimental Setup

The system was developed and tested in a standard development environment with the following configuration:

Hardware Requirements

- **Processor:** Minimum Intel i5 or equivalent
- **RAM:** 8 GB or higher
- **Storage:** 256 GB SSD recommended

Software Requirements

- **Frontend:** React (Vite + TypeScript), Material UI
- **Backend:** FastAPI (Python)
- **Database:** PostgreSQL
- **ORM & Migration:** SQLAlchemy, Alembic
- **Version Control:** Git
- **API Testing:** Swagger
- **Cloud Storage:** Cloudinary
- **Deployment:** Render

System Configuration

- **OS:** Windows
- **Python Environment:** Virtual environment (venv)
- **Node.js:** For frontend development

This setup ensured smooth development, testing, and deployment of the application.

4.3 Test Cases

To validate the functionality and internal logic of the QuickBite system, both black box and white box testing approaches were used. The following tables present structured test cases categorized by system operations.

4.3.1 Authentication and User Management Operations

Test ID	Scenario	Input	Expected Output	Actual Result
BB-01	User Registration	Valid name, email, password	User created, OTP sent	As Expected
BB-02	OTP Verification (Valid)	Correct OTP	Account verified	As Expected
BB-03	OTP Verification (Invalid)	Wrong OTP	400 Error (Invalid OTP)	As Expected
BB-04	OTP Expiry	Expired OTP	Error: OTP expired	As Expected
BB-05	Login Success	Valid credentials	JWT token returned	As Expected
BB-06	Login Failure	Wrong password	401 Unauthorized	As Expected

Table 2: Test Cases for Authentication and User Management Operations — black box tests covering registration, OTP verification, login success and failure

4.3.2 Cart Management Operations

Test ID	Scenario	Input	Expected Output	Actual Result
BB-07	Add to Cart	Valid food_id, quantity	Item added to cart	As Expected
BB-08	Add Invalid Food	Invalid food_id	Error response	As Expected
BB-09	Zero Quantity	quantity = 0	Validation error	As Expected
BB-10	Multiple Items	Different food items	All items stored correctly	As Expected

Table 3: Test Cases for Cart Management Operations — black box tests for adding items, invalid food IDs, zero quantity, and multiple items

4.3.3 Order Processing Operations

Test ID	Scenario	Input	Expected Output	Actual Result
BB-11	Place Order (Valid)	Cart with items	Order created (pending)	As Expected
BB-12	Empty Cart	No items	400 Error (Cart empty)	As Expected
BB-13	Duplicate Pending Order	Existing pending order	Request blocked	As Expected
BB-14	Invalid Token	Missing/invalid JWT	401 Unauthorized	As Expected

Table 4: Test Cases for Order Processing Operations — black box tests for valid order placement, empty cart, duplicate pending order, and invalid token

4.3.4 Admin Operations

Test ID	Scenario	Input	Expected Output	Actual Result
BB-15	Admin Approval	Valid order_id	Status updated to approved	As Expected
BB-16	Cart Clearance	Approved order	Cart cleared	As Expected
BB-17	Unauthorized Access	User tries admin API	403 Forbidden	As Expected
BB-18	Invalid Order ID	Non-existent order	404 Not Found	As Expected

Table 5: Test Cases for Admin Operations — black box tests for order approval, cart clearance, unauthorized access, and invalid order ID

4.3.5 Authentication Logic (White Box Testing)

Test ID	Code Area	Condition Tested	Expected Behavior	Result
WB-01	JWT Creation	Valid login	Token generated with payload	Passed
WB-02	JWT Validation	Valid token	Access granted	Passed
WB-03	JWT Expiry	Expired token	Unauthorized error	Passed
WB-04	Refresh Token	Valid cookie token	New access token generated	Passed

Table 6: White Box Test Cases for Authentication Logic — validating JWT creation, validation, expiry, and refresh token handling

4.3.6 OTP Verification Logic

Test ID	Condition	Code Path	Expected Output	Result
WB-05	OTP Match	user.otp == input	Verification success	Passed
WB-06	OTP Mismatch	user.otp != input	Error thrown	Passed
WB-07	OTP Expired	current_time > expiry	Expiry error	Passed

Table 7: White Box Test Cases for OTP Verification Logic — testing OTP match, mismatch, and expiry conditions

4.3.7 Order Logic Validation

Test ID	Condition	Code Logic	Expected Output	Result
WB-08	Pending Order Exists	if existing_order	Block request	Passed
WB-09	Empty Cart	if not cart_items	Raise error	Passed
WB-10	Price Calculation	loop over cart	Correct total computed	Passed
WB-11	Order Creation	valid flow	Order stored in DB	Passed

Table 8: White Box Test Cases for Order Logic Validation — verifying pending order check, empty cart handling, price calculation, and order creation

4.3.8 Admin Control Logic

Test ID	Condition	Code Logic	Expected Output	Result
WB-12	Valid Approval	status = approved	Order updated	Passed
WB-13	Cart Deletion	approved order	Cart cleared	Passed
WB-14	Email Trigger	approved order	Email sent	Passed
WB-15	Invalid Order	order not found	404 Error	Passed

Table 9: White Box Test Cases for Admin Control Logic — testing order approval, cart deletion, email trigger, and invalid order handling

4.3.9 Middleware and Authorization Logic

Test ID	Scenario	Condition	Expected Output	Result
WB-16	Valid Header	Bearer token present	User identified	Passed
WB-17	Missing Header	No Authorization	User = None	Passed
WB-18	Invalid Token	Decode fails	Unauthorized	Passed

Table 10: White Box Test Cases for Middleware and Authorization Logic — validating token presence, missing header, and invalid token scenarios

4.4 Safety Measures

Safety and security were important considerations during the development of the QuickBite system.

4.4.1 Authentication and Authorization

- JWT-based authentication ensures secure access
- Role-based access control restricts admin functionalities
- Protected routes prevent unauthorized operations

4.4.2 Data Security

- Passwords are stored in hashed form
- Sensitive data is not exposed in API responses
- Tokens are validated before processing requests

4.4.3 Input Validation

- Pydantic schemas validate all incoming data
- Prevents injection attacks and invalid inputs

4.4.4 API Security

- Middleware checks authentication headers
- Unauthorized access is blocked with proper error responses

4.4.5 Error Handling

- Proper HTTP status codes used
- Controlled error messages to avoid information leakage

4.4.6 Deployment Safety

- Secure handling of environment variables
- Cloud-based services used for reliability

CHAPTER – 5

RESULTS & DISCUSSION

5.1 Result Analysis

The QuickBite system was successfully developed as a full-stack web application integrating frontend, backend, and database layers. The results of the project are analyzed based on **functional correctness, system performance, reliability, and practical usability**.

5.1.1 Functional Outcome Analysis

The primary goal of the project was to implement a working food ordering system that reflects realworld workflows. The system successfully achieved the following functionalities:

- **User Authentication:**
 - Users can register using email and password, verify their account via OTP, and securely log in using JWT-based authentication. The use of token-based authentication ensured that only authorized users could access protected routes.
- **Role-Based Access Control:**
 - The system differentiates between users and admins. While users can browse food items and place orders, admins are responsible for managing and approving orders.
This separation of roles ensures structured system behavior.
- **Cart Management:**
 - Users can add food items to their cart with specified quantities. The cart acts as a temporary storage system before order placement, mimicking real-world e-commerce behavior.
- **Order Processing Workflow:**
 - The system implements a controlled workflow where:
 - Users place orders
 - Orders are initially marked as “Pending”

- Admin approves the order
- Cart is cleared upon approval
- Email notification is sent

- **Error Handling and Validation:**

The system effectively handles invalid inputs such as empty carts, duplicate pending orders, and invalid authentication tokens.

5.1.2 Performance Evaluation

Although the system was not tested under heavy load conditions, performance was evaluated under normal usage scenarios.

Metric	Observation	Interpretation
API Response Time	100–300 ms	Indicates efficient backend processing
Login Time	< 1 second	Fast authentication cycle
Order Placement Time	1–2 seconds	Includes validation and DB operations
Data Fetch Time	< 1 second	Efficient database querying
Deployment Stability	Stable	No crashes observed

Table 11: Performance Metrics of QuickBite System — observed response times for API calls, login, order placement, and data fetching

Analysis of Performance

The response times observed indicate that the system performs efficiently for small to moderate workloads. The use of FastAPI contributed to faster request handling, while PostgreSQL ensured reliable data retrieval.

The slight delay in order placement is expected due to multiple backend operations such as validation, price calculation, and database insertion.

5.1.3 System Reliability and Robustness

The system demonstrated strong reliability during testing:

- No unexpected crashes occurred
- API endpoints consistently returned correct responses
- Invalid inputs were handled gracefully
- Authentication failures were properly managed

Additionally, business logic constraints such as preventing multiple pending orders ensured system consistency.

5.1.4 Usability and User Experience

The frontend interface provides a structured and user-friendly experience:

- Clear navigation between pages
- Responsive UI using Material UI

5.1.5 Outcome of the Study

The results confirm that:

- A complete full-stack system can be developed using modern technologies
- Secure authentication and role-based workflows can be effectively implemented
- Real-world processes such as order management and notifications can be simulated

The project successfully bridges the gap between theoretical knowledge and practical implementation.

5.2 Comparison with Objectives

The objectives defined in Chapter 1 were systematically evaluated against the actual outcomes of the project.

Objective	Expected Outcome	Achieved	Remarks
Develop frontend system	Functional UI	Yes	React-based UI implemented
Build full-stack system	Integrated application	Yes	Backend + DB connected
Implement authentication	Secure login system	Yes	JWT + OTP implemented
Design REST APIs	API endpoints	Yes	Fully functional APIs
Apply modular design	Structured code	Yes	Services & interfaces used
Handle real-world logic	Business constraints	Yes	Pending order logic applied

Table 12: Comparison of Defined Objectives and Achieved Outcomes — mapping each project objective to its actual

result

5.3 Economic Analysis

5.3.1 Bill of Materials (Software Resources)

Component	Tool/Service	Cost	Purpose
Frontend	React (TypeScript)	Free	UI development

Backend	FastAPI	Free	API development
Database	PostgreSQL	Free	Data storage
ORM	SQLAlchemy	Free	DB interaction
Migration	Alembic	Free	Schema management
Testing	Swagger	Free	API testing
Version Control	Git	Free	Code management
Deployment	Render	Free tier	Hosting
Media Storage	Cloudinary	Free tier	Image handling

Table 13: Bill of Materials (Software Resources) — listing all tools and services used with their cost and purpose

5.3.2 Cost-Benefit Analysis

The system was developed entirely using open-source tools, resulting in minimal financial cost.

Benefits:

- No licensing fees
 - Easy deployment using cloud platforms
 - Scalability without significant infrastructure cost
- Cost Considerations (if scaled):**
- Paid cloud hosting

- Database scaling costs
- Third-party service subscriptions

5.3.3 Resource Utilization

- Efficient memory usage due to lightweight backend
- Optimized API calls reduce unnecessary load
- Cloud storage reduces local resource dependency

The system demonstrates efficient utilization of available resources.

5.4 Comparison with Existing Solutions and Standards

The QuickBite system was compared with industry-standard food delivery platforms and practices.

Feature	QuickBite	Industry Platforms	Standard Practice
Authentication	JWT-based	Advanced multi-factor	Secure token-based
UI/UX	Basic responsive	Highly optimized	Responsive UI
Order Workflow	Admin approval	Automated systems	Real-time processing
Payment Integration	Not implemented	Fully integrated	Essential feature
Scalability	Limited	High	Cloud scaling

Database	PostgreSQL	Distributed DB	Relational/NoSQL
Notifications	Email only	Push + SMS	Multi-channel

Table 14: Comparison of QuickBite with Existing Industry Solutions — evaluating authentication, UI, workflow, payment, scalability, and notifications

CHAPTER – 6

CONCLUSION & FUTURE SCOPE

6.1 Summary of Achievements

The QuickBite project represents a comprehensive effort to design and implement a full-stack web application that closely resembles real-world food ordering platforms. The project successfully integrates frontend, backend, and database technologies into a cohesive system, demonstrating both technical competence and practical understanding of software development principles.

One of the most significant achievements of this project is the successful implementation of a **layered and modular architecture**, where each component of the system operates independently while maintaining seamless communication. The frontend, developed using React with TypeScript, provides a structured and interactive user interface, while the backend, built with FastAPI, handles business logic, authentication, and data processing. The use of PostgreSQL as the database ensures reliable and structured data storage.

A key highlight of the system is the implementation of a **secure and scalable authentication mechanism**. The combination of JWT-based authentication and OTP-based email verification ensures that user access is both secure and verifiable. This reflects an understanding of real-world security practices, where multiple layers of authentication are often required.

Another important achievement is the development of a **realistic business workflow** within the application. The order lifecycle—from adding items to the cart, placing an order, validating constraints, and admin approval—demonstrates how real-world systems enforce rules and maintain control over operations. The inclusion of constraints such as preventing multiple pending orders shows attention to business logic and system integrity.

The project also emphasizes **clean coding practices and structured development**, including:

- Use of interfaces and service-based architecture in the frontend
- Separation of concerns in backend modules (routers, schemas, models)
- Use of ORM (SQLAlchemy) for database abstraction
- Schema validation using Pydantic

These practices contribute to the maintainability and scalability of the system.

From a learning perspective, the project provided hands-on experience in:

- Designing RESTful APIs
- Handling authentication and authorization flows
- Debugging real-world issues such as token errors and routing inconsistencies
- Integrating multiple technologies into a single application

Overall, the QuickBite system demonstrates the successful translation of theoretical knowledge into a functional and structured application, meeting all the objectives defined at the beginning of the internship.

6.2 Impact on Society, Environmental Sustainability, Ethical Issues, and Compliance

6.2.1 Societal Impact

The QuickBite application reflects the growing reliance on digital platforms for everyday services. Food delivery applications have become an essential part of modern lifestyles, especially in urban environments where convenience and time efficiency are critical.

The system contributes to societal advancement by:

- Providing a digital platform for food ordering, reducing the need for manual processes
- Enhancing user convenience through easy access to services
- Demonstrating how small-scale businesses can adopt digital solutions

Although the project is a prototype, it highlights the role of technology in improving service accessibility and operational efficiency. It also reflects how digital platforms can bridge the gap between service providers and consumers.

6.2.2 Environmental Sustainability

From an environmental perspective, the QuickBite system promotes sustainable practices by encouraging digital operations:

- Replacement of paper-based menus and receipts with digital interfaces
- Reduction in physical documentation through online record-keeping
- Use of cloud-based storage, minimizing the need for local infrastructure

Additionally, efficient backend design ensures that computational resources are utilized effectively, reducing unnecessary processing and energy consumption. While the environmental impact of a single application may be limited, the adoption of such systems at scale can contribute significantly to sustainability.

6.2.3 Ethical Considerations

Ethical responsibility is an important aspect of system design, particularly when dealing with user data and access control. The QuickBite system incorporates several ethical practices:

- Secure storage of passwords using hashing techniques
- Protection of user data through authentication mechanisms
- Restriction of access to sensitive operations using role-based control

The system ensures that user information is handled responsibly and that unauthorized access is prevented. Additionally, the implementation of OTP verification ensures that user identities are validated before granting access.

Ethical design also involves ensuring fairness and transparency in system behavior. The application enforces consistent rules for all users, such as preventing duplicate orders and validating inputs, which contributes to system integrity.

6.2.4 Compliance and Security Practices

While the project is not deployed in a regulated production environment, it follows several standard security practices:

- Validation of all incoming data using schemas
- Secure handling of authentication tokens
- Controlled access to protected routes

In a real-world scenario, additional compliance measures such as data protection regulations and privacy policies would need to be implemented. However, the current system provides a strong foundation for secure application development.

6.3 Limitations

Despite its successful implementation, the QuickBite system has certain limitations that should be acknowledged to provide a balanced evaluation.

6.3.1 Scalability Constraints

The system is designed for learning and demonstration purposes and has not been tested under high traffic conditions. As the number of users increases, performance optimization and infrastructure scaling would be required.

6.3.2 Lack of Payment Integration

The absence of a payment gateway limits the system's ability to function as a complete e-commerce solution. Payment integration is a critical component of real-world applications and would need to be implemented for practical deployment.

6.3.3 Limited Real-Time Functionality

The system does not support real-time updates such as live order tracking or instant notifications. These features are essential in modern applications and would enhance user experience.

6.3.4 Basic User Interface Design

While the interface is functional and responsive, it lacks advanced design features such as animations, personalization, and dynamic recommendations. Commercial applications typically invest heavily in UI/UX design to improve user engagement.

6.3.5 Deployment Limitations

The application is deployed using free-tier services, which may have restrictions on performance, uptime, and scalability. Production-level deployment would require more robust infrastructure.

6.3.6 Testing Limitations

Although functional and logical testing were performed, advanced testing techniques such as automated testing, load testing, and security audits were not conducted. These are essential for ensuring system reliability in real-world scenarios.

6.4 Future Work

The QuickBite system provides a strong foundation for further development and can be extended in multiple directions to enhance its functionality and scalability.

6.4.1 Integration of Payment Systems

Future versions of the application can include integration with payment gateways such as UPI, credit cards, or digital wallets. This would enable complete transaction processing and make the system suitable for real-world deployment.

6.4.2 Real-Time Communication and Tracking

Implementing real-time technologies such as WebSockets can enable features like:

- Live order tracking
- Instant status updates
- Real-time notifications

This would significantly improve user experience and system interactivity.

6.4.3 Advanced User Experience Enhancements

Enhancing the frontend with improved UI/UX design can include:

- Personalized recommendations
- Dynamic content rendering
- Improved navigation and animations

These features would make the application more engaging and user-friendly.

6.4.4 Scalability and Cloud Optimization

The system can be scaled using modern cloud technologies such as:

- Containerization (Docker)
- Orchestration (Kubernetes)
- Load balancing

This would allow the application to handle large user traffic efficiently.

6.4.5 AI and Data Analytics Integration

Machine learning techniques can be used to:

- Recommend food items based on user behavior
- Analyze order patterns

- Optimize system performance

This would add intelligence to the system and improve decision-making.

6.4.6 Enhanced Security Mechanisms

Future improvements can include:

- Multi-factor authentication (MFA)
- Rate limiting to prevent abuse
- Advanced encryption techniques

These enhancements would further strengthen system security.

6.4.7 Monitoring and Analytics Dashboard

Adding administrative dashboards for monitoring system performance and user activity would provide valuable insights and support data-driven decisions.

REFERENCES

- [1] Angular Team, “Angular documentation,” Angular Official Docs, 2025. [Online]. Available: <https://angular.dev/>
- [2] Meta Platforms, “React documentation,” React Official Docs, 2024. [Online]. Available: <https://react.dev/>
- [3] S. Ramírez, “FastAPI documentation” FastAPI Official Docs, 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [4] PostgreSQL Global Development Group, “PostgreSQL documentation,” 2024. [Online]. Available: <https://www.postgresql.org/docs/>
- [5] M. Bayer, “SQLAlchemy documentation,” 2024. [Online]. Available: <https://docs.sqlalchemy.org/>
- [6] SQLAlchemy Team, “Alembic migration tool documentation,” 2024. [Online]. Available: <https://alembic.sqlalchemy.org/>
- [7] Postman Inc., “Postman learning center,” 2024. [Online]. Available: <https://learning.postman.com/>
- [8] Microsoft, “TypeScript documentation,” 2024. [Online]. Available: <https://www.typescriptlang.org/docs/>
- [9] J. Dean and S. Ghemawat, “Design of scalable web applications,” *IEEE Softw.*, vol. 38, no. 2, pp. 45–52, 2021.
- [10] A. Lakshman and P. Malik, “Cassandra: A decentralized structured storage system,” IEEE Computer Society, 2020.
- [11] N. Dragoni *et al.*, “Microservices: Yesterday, today, and tomorrow,” *IEEE Softw.*, vol. 37, no. 3, pp. 64–72, 2020.
- [12] OWASP Foundation, “OWASP top 10: Web application security risks,” 2021. [Online]. Available: <https://owasp.org/>
- [13] Amazon Web Services, “Cloud architecture best practices,” 2023. [Online]. Available: <https://aws.amazon.com/architecture/>
- [14] Google Cloud, “Building scalable web applications,” 2022. [Online]. Available: <https://cloud.google.com/>
- [15] IETF, “JSON Web Token (JWT) Standard (RFC 7519),” 2020. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7519>
- [16] S. Newman, *Building Microservices*, 2nd ed. O’Reilly Media, 2021.
- [17] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2021.

- [18] Cloudinary Inc., “Cloudinary documentation,” 2024. [Online]. Available: <https://cloudinary.com/documentation>
- [19] Render Inc., “Render deployment documentation,” 2024. [Online]. Available: <https://render.com/docs>
- [20] Medium, “Why TypeScript improves code quality in large applications,” 2023. [Online]. Available: <https://medium.com/>

ANNEXURE

Sustainability Statement:

The QuickBite project has been designed with consideration for its broader impact on society, the environment, and ethical use of technology. While the application is primarily a software system, its design and functionality reflect principles of sustainability and responsible development.

Environmental Sustainability The system promotes environmentally sustainable practices by encouraging digital interaction over traditional physical processes. By replacing printed menus, paper receipts, and manual order handling with a digital platform, the application reduces paper consumption and associated waste. Additionally, the use of cloud-based infrastructure minimizes the need for physical hardware resources, contributing to more efficient energy utilization. Efficient backend design and optimized API calls further ensure that computational resources are used effectively, reducing unnecessary processing and energy consumption.

Social Sustainability From a societal perspective, QuickBite improves accessibility and convenience by providing users with an easy-to-use digital platform for ordering food. It demonstrates how technology can simplify everyday tasks and improve service delivery. The system also reflects real-world business operations, showing how digital platforms can support small businesses in reaching a wider audience. By enabling structured order management and administrative control, the application highlights the role of technology in improving operational efficiency.

Ethical Considerations Ethical responsibility has been incorporated into the system through secure handling of user data and controlled access mechanisms. Sensitive information such as passwords is stored in hashed form, and authentication is enforced using secure token-based methods. The system ensures fairness by implementing role-based access control, preventing unauthorized users from performing administrative actions. Input validation mechanisms are used to prevent misuse and ensure that the system operates reliably.

Harshit Tyagi

22103290 Intern at Evalueserve Pvt. Ltd.

 Quick Submit Quick Submit Jaypee Institute of Information Technology

Document Details

Submission ID

trn:oid::1:3547648634

Submission Date

Apr 23, 2026, 1:47 PM GMT+5:30

Download Date

Apr 23, 2026, 2:07 PM GMT+5:30

File Name

ht.pdf

File Size

1.6 MB

61 Pages

9,722 Words

60,495 Characters

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 14 words)

Match Groups

-  **17 Not Cited or Quoted 6%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 4%  Internet sources
- 0%  Publications
- 6%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.